

# Week 3

# Revision

# Guide

Core ML · Days 17 – 23

*Concepts · Code · Real World Problems · Interview Prep*

| WHAT'S INSIDE |                                            |
|---------------|--------------------------------------------|
| DAY 16 Recap  | Week 2 one-line recap before Week 3 begins |
| DAY 17        | Random Forests                             |
| DAY 18        | Overfitting vs Underfitting                |
| DAY 19        | Model Evaluation                           |
| DAY 20        | Cross Validation & Hyperparameter Tuning   |
| DAY 21        | Gradient Boosting (XGBoost)                |
| DAY 22        | K-Means Clustering                         |
| DAY 23        | Dimensionality Reduction (PCA)             |
| DAY BONUS     | MNC Interview Q&A;                         |

*Interview Questions · Code Syntax · Real World Examples #42DaysOfML*

## DAY 16 RECAP · BEFORE WE BEGIN

Week 2 built feature work and the core ML loop. Here is the one-line summary of each day, so Week 3 builds on solid ground.

| Day | Topic               | One-Line Takeaway                                                                                                            |
|-----|---------------------|------------------------------------------------------------------------------------------------------------------------------|
| 09  | Feature Engineering | Raw columns hide signal. Decompose timestamps, bin continuous features, create interaction terms. Measure before/after.      |
| 10  | Feature Selection   | More features is not always better. Drop near-zero-variance and high-correlation first, then use RFE or Lasso.               |
| 11  | Scaling & Encoding  | KNN/SVM break without scaling. Fit StandardScaler on train only. Label encode targets only; one-hot or ordinal for features. |
| 12  | How ML Works        | Every model runs the same 4-step loop: predict -> measure loss -> compute gradient -> step downhill. Repeat.                 |
| 13  | Linear Regression   | Coefficients become meaningless when features are correlated. RMSE >> MAE signals a few very large errors.                   |
| 14  | Logistic Regression | Sigmoid squashes output to (0,1). Default threshold 0.5 is a choice, not a law. Check recall on the minority class.          |
| 15  | Decision Trees      | 100% train accuracy is a red flag. Constrain max_depth to 3-6. Gini vs entropy rarely changes the result.                    |

## WEEK 3 ROADMAP

Week 2 ended with Decision Trees. Week 3 stacks on top: Random Forests (bagging), Overfitting vs Underfitting (the unifying lens for all model complexity), Model Evaluation beyond accuracy, Cross Validation and Hyperparameter Tuning, XGBoost (boosting), K-Means Clustering (the first unsupervised algorithm in the series), and PCA (dimensionality reduction). By Day 23, you have the full supervised ML toolkit and can explain every decision to an interviewer.

## SELF-CHECK BEFORE STARTING WEEK 3

Without notes: explain why 100% training accuracy on a Decision Tree is a warning sign. State what the sigmoid function does and why linear regression fails for classification. Describe the 4-step gradient descent loop. If any feel shaky, revisit that day before continuing.

## DAY 17 · RANDOM FORESTS

### THE CORE IDEA

A single Decision Tree overfits because it memorises the specific noise in its training data. A Random Forest builds many trees, each trained slightly differently, and averages their votes. Each tree overfits in a **different** way. Averaging those differences cancels them out — this is the entire idea behind bagging.

### BAGGING: TWO SOURCES OF RANDOMNESS

| Mechanism              | What It Does                                                                                                                           | Why It Matters                                                                       |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Bootstrap Sampling     | Each tree trains on a random sample of rows drawn <b>with replacement</b> . About 63% of rows are used; 37% are left out (out-of-bag). | Each tree sees different rows. Mistakes on row X affect Tree 1 but maybe not Tree 2. |
| Random Feature Subsets | At each split, only a random subset of features is considered, not all of them.                                                        | Trees are decorrelated. One strong feature does not dominate every tree.             |

### FROM THE DAY 17 NOTEBOOK — VARIANCE REDUCTION

Single Decision Tree across 10 random seeds: mean accuracy 66.1%, std dev ~2.1 pts.  
Random Forest (100 trees) across the same 10 seeds: mean accuracy 77.8%, std dev ~1.7 pts.  
The forest does not just raise the mean — it narrows the spread. Reported accuracy is something you can actually trust, not an artifact of which random seed you happened to use.

### KEY SYNTAX

```
from sklearn.ensemble import RandomForestClassifier

# Unconstrained forest – averaging over 100 trees reduces overfitting
# even without a depth limit
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

# Feature importance is a free byproduct – averaged across all 100 trees
# (more stable than a single tree's importance)
for name, imp in zip(feature_names, rf.feature_importances_):
    print(f"{name}: {imp:.4f}")

# Gains level off around 100-300 trees on most datasets.
# Going beyond rarely helps and just adds training time.
```

### THE PRODUCTION TRAP

Two engineers on the same team both build a fraud detection model using a single Decision Tree. Same data, different random seeds. One reports 68.8% test accuracy, the other reports 62.5%. A 6-point gap sparks a heated model review debate about whose feature engineering is better. Neither engineer did anything wrong. The variance is intrinsic to using one tree. Fix: switch both to Random Forest. The std dev drops and mean accuracy jumps. If your model's reported accuracy depends heavily on random\_state, that's the algorithm, not your data.

### GOLDEN RULE

n\_estimators=100-300 is a sensible production default. Feature importances from a Random Forest are more stable than from a single tree because they are averaged across bootstrap samples — use them, but still validate with SHAP for individual predictions (see Day 21).

## DAY 18 · OVERFITTING vs UNDERFITTING

### THE CORE IDEA

A model that scores 99% on training data and 60% on new data has not learned the pattern — it has memorised the training set. The bias-variance tradeoff is the unifying framework for understanding every model complexity decision in the series so far.

| State                       | What It Means                                 | How To Spot It                                       |                                                                 |
|-----------------------------|-----------------------------------------------|------------------------------------------------------|-----------------------------------------------------------------|
| Underfitting (High Bias)    | Model too simple to capture the real pattern. | Low train AND val accuracy. Small gap between them.  | Increase model complexity, add features, reduce regularization. |
| Good Fit                    | Captures signal without memorising noise.     | Both train and val accuracy high and close together. | Ship it. Monitor for drift.                                     |
| Overfitting (High Variance) | Memorises training noise along with signal.   | Very high train, meaningfully lower val. Large gap.  | Add regularization, constrain depth, get more data.             |

### FROM THE DAY 18 NOTEBOOK — REAL NUMBERS (Breast Cancer Dataset)

max\_depth=1: train=74.4%, val=78.8% — underfitting (both low, shallow gap)  
max\_depth=3: train=83.4%, val=85.0% — good fit  
max\_depth=5: train=90.3%, val=71.3% — overfitting starts (19-pt gap)  
max\_depth=10+: train=100.0%, val=63.7% — severe overfitting (36-pt gap)  
A 100% training score here is a WARNING SIGN, not a success.

### LEARNING CURVES — THE TOOL

**Reading a learning curve:** plot train score and validation score as you increase the amount of training data. An overfit model holds a near-perfect train score while the validation score lags far below it — the red shaded gap in the Day 18 notebook. More data helps close this gap. An underfit model shows both curves close together but both low — more data alone will not fix it, the model needs more capacity.

```
from sklearn.model_selection import learning_curve
import numpy as np

train_sizes, train_scores, val_scores = learning_curve(
    model, X, y, cv=5,
    train_sizes=np.linspace(0.1, 1.0, 8),
    scoring="accuracy", random_state=42
)
train_mean = train_scores.mean(axis=1)
val_mean = val_scores.mean(axis=1)
# Large persistent gap = overfit. Both low = underfit.
```

### REGULARIZATION FIX

Ridge (L2) regularization adds a penalty for large coefficients, shrinking them toward zero without dropping features entirely. From the Day 18 notebook on a degree-15 polynomial (severely overfit without regularization):

| Ridge alpha      | Effect                                    |
|------------------|-------------------------------------------|
| alpha=0 (no reg) | High train R2, low test R2 — overfit      |
| alpha=0.001      | Gap narrows sharply, test R2 jumps        |
| alpha=0.01       | Best balance — train and test R2 close    |
| alpha too high   | Both drop — tipped back into underfitting |

### GOLDEN RULE

Diagnosis order: (1) plot train vs val accuracy across model complexity. (2) If large gap: add regularization or constrain depth. (3) If both low: increase capacity or add features. Reading a learning curve takes 3 lines of code and removes the guesswork.

## DAY 19 · MODEL EVALUATION

### THE CORE IDEA

Accuracy answers one question: out of all predictions, how many were correct? That sounds like the whole story — until your classes are imbalanced. The right metric depends on what a false positive costs you versus what a false negative costs you.

| Metric    | Formula                  | Measures                                |                                                                                 |
|-----------|--------------------------|-----------------------------------------|---------------------------------------------------------------------------------|
| Accuracy  | $(TP+TN) / \text{Total}$ | Overall: how many correct?              | Lies on imbalanced data. 95%-class dataset scores 95% predicting only majority. |
| Precision | $TP / (TP+FP)$           | Of flagged positives, how many right?   | Optimise when false alarms costly — spam filters.                               |
| Recall    | $TP / (TP+FN)$           | Of all real positives, how many caught? | Optimise when missed cases costly — cancer, fraud.                              |
| F1        | $2*P*R / (P+R)$          | Harmonic mean of P and R.               | Single number when both matter. Hides which one is failing.                     |
| ROC-AUC   | Area under ROC curve     | Ranking quality across all thresholds.  | Threshold-independent. 0.5 = random, 1.0 = perfect.                             |

### FROM THE DAY 19 NOTEBOOK — THE ACCURACY PARADOX (95% imbalanced dataset)

DummyClassifier (always predicts majority, learns nothing): Accuracy=94.7%, Recall=0%  
LogisticRegression (default settings): Accuracy=94-95%, Recall=3% — nearly identical!  
LogisticRegression (class\_weight="balanced"): Accuracy drops to ~72%, Recall jumps to ~75%.  
That drop in accuracy is NOT the model getting worse. It is the model finally doing its job.

### THE THRESHOLD IS A BUSINESS DECISION

The default 0.5 threshold is not a law — it is a starting point. Lowering it flags more cases as positive: recall goes up, precision goes down, more false alarms. The right threshold depends on what a missed case costs versus what a false alarm costs.

```
# Get probabilities, not just class labels
y_proba = model.predict_proba(X_test)[: , 1]

# Try multiple thresholds – this is a business decision, not a model one
for t in [0.1, 0.2, 0.3, 0.5]:
    preds = (y_proba >= t).astype(int)
    print(t, precision_score(y_test, preds), recall_score(y_test, preds))

# ROC-AUC: model quality across every possible threshold
from sklearn.metrics import roc_auc_score
print(roc_auc_score(y_test, y_proba)) # threshold-independent
```

### THE PRODUCTION TRAP

A fraud model reports 95% accuracy. The manager celebrates.  
Recall on the fraud class: 3%. The model catches almost nothing.  
95% of transactions are legitimate — predicting "not fraud" for everything scores 95%.  
Fix: check class\_weights in your confusion matrix BEFORE reporting accuracy to anyone.  
In a regulated environment, a model with 95% accuracy and 3% recall will not pass audit.

### GOLDEN RULE

Match the metric to the cost of being wrong. Cancer screening and fraud detection optimise recall — missing a case is expensive. Spam filters optimise precision — a false alarm (legitimate email in spam) costs more than missing spam. ROC-AUC tells you whether the model knows something useful. The threshold decides whether you act on it.

## DAY 20 · CROSS VALIDATION & HYPERPARAMETER TUNING

### THE CORE IDEA

A single train/test split tells you how a model did on one particular slice of data. Shuffle the split differently and the same model, same data, same code can report a meaningfully different score. That is not your model changing. That is noise.

| Technique          | What It Does                                                                          | When To Use It                                                                      |
|--------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| K-Fold CV          | Split data into K folds. Train on K-1, validate on the held-out fold. Repeat K times. | Every row gets one turn as validation data. Average K scores for a stable estimate. |
| GridSearchCV       | Try every combination of hyperparameters you specify, scored with CV.                 | Exhaustive search. Correct when you have a good idea of the useful range.           |
| RandomizedSearchCV | Sample a fixed number of random combinations from a continuous distribution.          | Right when the space is too large to grid out, or when you are exploring blind.     |
| StratifiedKFold    | K-Fold that preserves class balance across every fold.                                | Required whenever classes are imbalanced (always pair with Day 19 evaluation).      |

### FROM THE DAY 20 NOTEBOOK — ONE SPLIT IS NOT ENOUGH

Same Random Forest, same data, same code. Only the random\_state for the split changes.  
Accuracy across 10 seeds: ranges from 77.8% to 88.9% — an 11-point swing.  
K-Fold CV (5 folds, same model): 83.4% +/- 2.8 pts — one trustworthy number.  
GridSearchCV on SVC: default CV score 0.823 -> tuned CV score 0.861 (+3.8 pts).  
The only change was searching the hyperparameter space. Same model class, same data.

### THE LEAKAGE TRAP — THE MOST COMMON CV MISTAKE

Anything that **learns from data** — scalers, feature selectors, imputers — must live inside a Pipeline and refit on each training fold separately. The moment a preprocessing step sees the full dataset before CV starts, your validation score is no longer a validation score.

```
# WRONG: selector fitted on full dataset BEFORE cross-validation
selector = SelectKBest(k=10).fit_transform(X, y) # sees all labels!
cv_scores = cross_val_score(SVC(), selector, y, cv=5) # score is inflated

# CORRECT: selector inside Pipeline, refits on each training fold only
from sklearn.pipeline import Pipeline
pipe = Pipeline([
    ("select", SelectKBest(k=10)),
    ("svc", SVC()),
])
cv_scores = cross_val_score(pipe, X, y, cv=5) # honest score

# Same rule applies to: StandardScaler, imputers, target encoders.
# Wrap them all in a Pipeline.
```

### THE PRODUCTION TRAP

From the Day 20 notebook: 1,000-feature dataset, only 3 informative features, 100 rows.  
Leaky CV score (feature selection BEFORE CV): ~0.73  
Correct CV score (feature selection INSIDE Pipeline): ~0.55  
An 18-point inflated score from leakage. The model looks like it works. It does not.  
In production, this shows up as a model that aced evaluation and failed on new data.  
Rule: if it has a .fit() method and learns from your data, it belongs in the Pipeline.

### GOLDEN RULE

learning\_rate and n\_estimators are a pair — never tune one without considering the other. Use RandomizedSearchCV when the parameter space is continuous or too large to exhaustively grid. Always keep a final held-out test set you never touch during CV or tuning — that is your honest final score.



## DAY 21 · GRADIENT BOOSTING (XGBoost)

### BAGGING vs BOOSTING

|                     | Random Forest (Bagging)                          | XGBoost (Boosting)                                               |
|---------------------|--------------------------------------------------|------------------------------------------------------------------|
| Construction        | Trees built in parallel, independently           | Trees built sequentially — each corrects the previous            |
| What each tree sees | A bootstrap sample — random rows                 | The residual errors from all previous trees                      |
| Variance vs Bias    | Reduces variance (averaging cancels overfitting) | Reduces bias (sequential correction improves fit round by round) |
| Overfitting risk    | Lower — averaging dilutes individual mistakes    | Higher — can chase noise if too many rounds or high lr           |
| Missing values      | No native handling — impute first                | Handles natively — learns best direction for missing values      |
| When it wins        | Noisy data, mislabeled rows, fast baselines      | Structured/tabular data, Kaggle competitions, max accuracy       |

### FROM THE DAY 21 NOTEBOOK — WATCHING BOOSTING BOOST (loss per round)

XGBClassifier trained with eval\_set logging loss at every boosting round:  
After tree 1: train loss=0.6217, test loss=0.6224  
After tree 50: train loss=0.3891, test loss=0.4012  
After tree 100: train loss=0.2864, test loss=0.4109 <-- test starts creeping back up  
Train loss keeps dropping the entire time. Test loss flattens then rises slightly after ~60 rounds.  
That uptick is overfitting appearing one tree at a time. This is why early\_stopping\_rounds exists.

On the same dataset:  
Random Forest (100 trees): CV = 87.1% +/- 1.8 pts  
XGBoost (100 trees): CV = 88.3% +/- 1.4 pts  
17 of 29 Kaggle competition winning solutions in 2015 used XGBoost (Chen & Guestrin paper).

### KEY SYNTAX

```
from xgboost import XGBClassifier
import shap

# Train with early stopping — stops adding trees once test loss plateaus
model = XGBClassifier(n_estimators=500, learning_rate=0.05,
                      max_depth=3, random_state=42, eval_metric="logloss")
model.fit(X_train, y_train,
          eval_set=[(X_test, y_test)],
          early_stopping_rounds=10, verbose=False)

# SHAP: explain individual predictions, not just global importance
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test[:200])
shap.summary_plot(shap_values, X_test[:200]) # direction + magnitude per row
```

### LEARNING RATE IS A PAIR WITH n\_estimators

| learning_rate / n_estimators | Effect at n_estimators=100 fixed                                    |
|------------------------------|---------------------------------------------------------------------|
| lr=0.01, n=100               | Underperforms — 100 small steps have not got far enough             |
| lr=0.1, n=100                | Solid default — reasonable speed of correction                      |
| lr=0.5, n=100                | Often overshoots — aggressive corrections can overshoot the minimum |
| lr=0.01, n=500               | Often matches or beats lr=0.1, n=100 — just needs more rounds       |

#### GOLDEN RULE

`feature_importances_` tells you average impact across the dataset. SHAP tells you why one specific prediction came out the way it did, and whether a feature pushed it up or down. Use `feature_importances_` for a quick overview. Use SHAP when a stakeholder or regulator asks "why did the model flag this case specifically."

## DAY 22 · K-MEANS CLUSTERING

### THE CORE IDEA

K-Means is the first unsupervised algorithm in this series — no labels needed. It groups data points into K clusters by repeatedly assigning each point to its nearest centroid, then updating the centroid to the mean of its cluster. Repeats until centroids stop moving.

| Concept          | What It Is                                                                                                                | What It Tells You                                                                                                    |
|------------------|---------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| K-Means Loop     | Init K centroids. Assign each point to nearest centroid. Update centroid to mean of its points. Repeat until convergence. | Use <code>n_init=10</code> — convergence is guaranteed but not to the global optimum. Multiple starts pick the best. |
| WCSS (Inertia)   | Within-Cluster Sum of Squares: total squared distance of each point from its centroid.                                    | Lower = tighter clusters. Use to find the elbow.                                                                     |
| Silhouette Score | How similar each point is to its own cluster vs the nearest other cluster. Range: -1 to +1.                               | +1 = well-separated. 0 = on boundary. Negative = wrong cluster.                                                      |

### THE ELBOW METHOD + SILHOUETTE SCORE — CHOOSING K

Neither method gives you a definitive single answer — they are guides. The elbow is where WCSS stops dropping sharply. The silhouette score peaks at the most well-separated K. Use both, then sanity-check by looking at what each cluster actually represents.

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# Always scale before K-Means – distance-based algorithm
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Elbow method
wcss = []
for k in range(1, 11):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X_scaled)
    wcss.append(model.inertia_)

# Silhouette score (needs k >= 2)
for k in range(2, 8):
    labels = KMeans(n_clusters=k, random_state=42, n_init=10).fit_predict(X_scaled)
    print(k, silhouette_score(X_scaled, labels))
```

### REAL WORLD — CUSTOMER SEGMENTATION (Mall Customer Dataset)

K-Means on Annual Income and Spending Score identifies 5 natural customer segments:  
High income / high spending (premium targets), High income / low spending (savers),  
Low income / high spending (impulsive buyers), Low income / low spending (budget shoppers),  
Mid income / mid spending (average customers).  
Starbucks, Swiggy, and Flipkart all use cluster-based segmentation for targeted offers.  
The model finds the groups — the business team names them and designs the campaigns.

### THE PRODUCTION TRAP

K-Means is a distance-based algorithm. Features on different scales break it silently.  
Income (range 0-150,000) will dominate Spending Score (range 1-100) without scaling.  
The clusters will reflect income alone, not the true pattern.  
Fix: always `StandardScaler` before fitting. Fit on train only if you have a train/test split.  
Second trap: K-Means assumes spherical clusters of similar size.  
If your clusters are elongated, nested, or very different in density, use DBSCAN instead.

#### GOLDEN RULE

Silhouette score range: +1 = excellent separation, 0 = clusters overlap, negative = points assigned to wrong cluster. A silhouette score above 0.5 is generally considered a strong clustering result. Always visualise at least 2 features after clustering to sanity-check that the groups make domain sense.

## DAY 23 · DIMENSIONALITY REDUCTION (PCA)

### THE CORE IDEA

As the number of features increases, data becomes sparse and models become harder to train — this is the curse of dimensionality. PCA (Principal Component Analysis) creates new features called Principal Components that are linear combinations of the original features, ordered so the first component captures the most variance, the second captures the next most, and so on. Components are orthogonal — no redundancy between them.

| Rule               | Why It Matters                                                                                                             | Production Implication                                                                               |
|--------------------|----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Scale First        | PCA finds directions of maximum variance. A feature with range 0-130,000 will dominate one with range 0-1 purely by scale. | Always StandardScaler before PCA. Without it, PCA reflects scale, not information.                   |
| Explained Variance | How much of the original data's total variance is captured by each component.                                              | 95% explained variance is the standard production threshold for choosing <code>n_components</code> . |
| Scree Plot         | Plot cumulative explained variance vs number of components.                                                                | Pick the "elbow" — the point where the curve flattens. Each component after that adds little.        |
| Fit on Train Only  | PCA is a transformation that learns from data (the covariance matrix).                                                     | Fit PCA on training data only. Transform test using the same fitted PCA. Data leakage otherwise.     |

### FROM THE DAY 23 NOTEBOOK — BREAST CANCER DATASET (30 features)

Components needed to explain 95% variance: 10 (out of 30) — 67% fewer features.  
Components needed to explain 99% variance: 17 (out of 30).

Accuracy comparison (LogisticRegression):

All 30 original features: 97.37%

10 PCA components (95% var): 98.25% <-- MORE accurate with fewer features

2 PCA components (44% var): lower <-- too much information lost

Feature pairs with correlation > 0.9: 17 pairs out of 30 features.

These redundant features add noise without adding information.

PCA removes the redundancy and the model generalises better.

### KEY SYNTAX — PRODUCTION-READY PCA IN A PIPELINE

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

# Find the right n_components first
pca_full = PCA().fit(X_train_scaled)
cumvar = np.cumsum(pca_full.explained_variance_ratio_)
n_95 = np.argmax(cumvar >= 0.95) + 1 # components for 95% variance

# Production pipeline: leakage-proof, serializable with joblib
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=n_95)),
    ("model", LogisticRegression(max_iter=1000))
])
pipe.fit(X_train, y_train) # scaler + PCA fit on train only, automatically
pipe.predict(X_test)      # same transformations applied, no leakage
```

### WHEN TO USE PCA AND WHEN NOT TO

|                  | Details                                                                                                                                                                                                                   |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| USE PCA when:    | Many correlated features (multicollinearity). Training slow due to feature count. Visualising high-dimensional clusters in 2D. Preprocessing before distance-based models (KNN, K-Means, SVM).                            |
| DO NOT USE when: | You need to explain which original features drive predictions (PCA destroys interpretability). Tree-based models (Random Forest, XGBoost) handle high dimensions natively. Non-linear relationships — PCA is linear only. |

## THE PRODUCTION TRAP

Fitting PCA on the full dataset before splitting is data leakage — same rule as Day 20.  
 PCA components are not interpretable. "PC1 has a high loading for mean radius and mean perimeter" is not an explanation you can give a compliance officer or a product manager.  
 If interpretability matters, use feature selection (Day 10) instead of PCA.  
 Netflix reduced a 500-feature user preference matrix to 20 PCA components in their early recommendation system — not for interpretability, but for speed and generalisation.

## GOLDEN RULE

PCA is not a feature selection method — it does not keep original features, it creates new ones. The new components are linear combinations of everything. Always plot the scree curve before deciding n\_components. 95% explained variance is a production-tested threshold that balances compression with information retention.

## WEEK 3 · ALL INTERVIEW QUESTIONS AT A GLANCE · MNC INTERVIEW Q&A;

*Random Forests · Overfitting · Model Evaluation · Cross Validation · XGBoost · K-Means · PCA*

### DAY 17 · Q1 GENERAL MLE ROUND

Your team reports two different accuracies for the same model trained on the same data. What is the most likely cause?

Single Decision Tree with different random seeds or train/test splits.  
The tree's reported accuracy is high-variance — highly dependent on which specific splits it locks onto.  
Fix: switch to Random Forest. A 100-tree forest trained across 10 seeds shows far lower std dev than a single tree.  
Report CV mean +/- std instead of a single split score whenever comparing models.

### DAY 17 · Q2 FLIPKART — ML ENGINEER ROUND

Why does a Random Forest generalise better than an unconstrained single Decision Tree, even without setting max\_depth?

Bagging. Each tree trains on a bootstrap sample (different rows, with replacement) and considers only a random subset of features at each split. Each tree overfits in a different direction.  
Majority voting averages out the individual mistakes. No one tree's noise dominates the final prediction.  
Key detail: even an unconstrained Random Forest generalises better than an unconstrained single tree — bagging reduces variance without requiring depth constraints, though max\_depth still helps both.

### DAY 18 · Q1 GENERAL MLE ROUND

A model has 100% training accuracy and 63% test accuracy. What is happening and how do you fix it?

Classic overfitting — the model memorised the training data, including noise, rather than learning the pattern.  
The large train-test gap (37 points) is the diagnostic signal. Plot a learning curve to confirm.  
Fixes: constrain max\_depth (to 3-6 for trees), add L2 regularization (Ridge alpha), get more training data, or use a Random Forest to reduce variance through bagging. Do not just lower the learning rate — diagnose first.

### DAY 18 · Q2 GENERAL MLE ROUND

How do you use a learning curve to decide whether to collect more data or change the model?

Plot train and validation scores as training set size increases.  
If the gap between train and val is large and stays large as data grows: model is overfitting.  
More data helps narrow the gap but does not eliminate it — the model itself also needs to change (regularize or simplify).  
If both curves are close together but both low: model is underfitting.  
More data will not help — the model lacks the capacity to learn the pattern. Increase complexity instead.

### DAY 19 · Q1 PAYTM — DATA SCIENTIST ROUND

Your fraud detection model has 98% accuracy. The business team is satisfied. What do you do next?

Check the class distribution. If 98% of transactions are legitimate, a model predicting "not fraud" for everything also scores 98% accuracy — it may be learning nothing useful.  
Look at the confusion matrix and compute recall on the fraud class specifically.  
If recall is near 0, the model is useless regardless of overall accuracy.  
Discuss the threshold with the business team: for fraud, a missed case costs far more than a false alarm.  
A lower threshold (e.g., 0.1 instead of 0.5) may catch more fraud at the cost of more false alerts — a business decision, not a model one.

### DAY 19 · Q2 GENERAL MLE ROUND

When would you choose recall over precision, and when the opposite? Give one example of each.

Recall when a false negative is expensive: cancer screening (missing a tumour is worse than an extra test), fraud detection (a missed fraud costs more than investigating a false alarm).  
Precision when a false positive is expensive: spam filtering (moving a real email to spam is worse than letting a spam email through), content moderation (falsely flagging legitimate content has reputational cost).  
F1 is a compromise — use it when both types of error matter roughly equally and you want one number.  
ROC-AUC is threshold-independent — use it when comparing model quality across operating points.

### DAY 20 · Q1 GOOGLE — ML ENGINEER ROUND

You are evaluating two models using 5-fold CV. Model A scores 0.91 +/- 0.08, Model B scores 0.89 +/- 0.01. Which do you deploy?

Model B. The standard deviation matters as much as the mean.  
Model A's 0.08 std means its performance swings up to ~0.99 or down to ~0.83 across folds.  
That variance suggests it is sensitive to which specific data it sees — less reliable in production.  
Model B's consistent 0.89 +/- 0.01 will likely perform close to that estimate on new data.  
In production, predictability is often more valuable than a slightly higher mean that comes with high variance.

#### DAY 20 · Q2 SWIGGY — ML ENGINEER ROUND

Describe exactly how to tune a model with GridSearchCV without introducing data leakage.

Split train/test first. The test set is never touched until the final evaluation.  
Wrap all preprocessing (scaling, feature selection, imputation) inside a sklearn Pipeline.  
Pass the Pipeline — not preprocessed data — to GridSearchCV. CV refits preprocessing on each fold.  
Use the best `_estimator_` from GridSearchCV to predict on the held-out test set.  
The test score from the held-out set is your honest final number. The CV score is the tuning score.  
Never re-use the test set for another round of tuning — that re-introduces leakage.

#### DAY 21 · Q1 AMAZON — ML ENGINEER ROUND

Your XGBoost model's test loss starts rising after round 60 even though train loss keeps falling. What do you do?

This is overfitting by boosting round. Each new tree is correcting errors on data the previous trees memorised.  
Fix: use `early_stopping_rounds=10` (stop adding trees when test loss has not improved for 10 consecutive rounds).  
This typically stops training automatically around the round where test loss was at its minimum.  
Also consider: lower `n_estimators`, increase regularization (`reg_alpha`, `reg_lambda`), or lower `learning_rate` paired with higher `n_estimators` + early stopping for a better generalisation tradeoff.

#### DAY 21 · Q2 GENERAL MLE ROUND

When would you pick Random Forest over XGBoost? Give a specific scenario.

Noisy data or mislabeled rows. Boosting treats errors as "mistakes to correct" round after round, so it chases noise aggressively. Bagging averages over many trees, diluting the influence of any single bad row.  
Also: when you need a fast, reliable baseline with minimal tuning. Random Forest's defaults are solid.  
Random Forest also handles high missing-value rates less gracefully than XGBoost (which handles missing natively).  
In regulated domains where explainability is required, Random Forest's individual trees are easier to reason about.  
For maximum accuracy on clean, structured data with time to tune: XGBoost or LightGBM.

#### DAY 22 · Q1 ZOMATO — DATA SCIENTIST ROUND

You ran K-Means with K=5. How do you know the clusters are meaningful and not just a mathematical artifact?

Three checks: (1) Silhouette score — if above 0.5, clusters are well-separated. If near 0 or negative, points are on cluster boundaries or in the wrong cluster.  
(2) Visualise — plot two features coloured by cluster label. If clusters overlap visually, the structure is weak.  
(3) Domain sense — do the clusters have a meaningful business interpretation?  
A "high income / high spending" cluster has an actionable meaning. A random split does not.  
K-Means will always find K groups in any data — the job is deciding whether those groups are real.

#### DAY 22 · Q2 GENERAL MLE ROUND

Why do you scale features before K-Means? What breaks if you do not?

K-Means computes Euclidean distance between every point and every centroid at each iteration.  
Without scaling, a feature with range 0-130,000 (income) dominates a feature with range 1-100 (spending score).  
The model effectively clusters on income alone, ignoring spending score entirely.  
StandardScaler puts every feature at mean=0, std=1 — each contributes equally to the distance calculation.  
Fit the scaler on training data only, same rule as for supervised models.

#### DAY 23 · Q1 META — ML ENGINEER ROUND

You have a dataset with 500 features. A colleague suggests PCA down to 20 components before modelling. Do you agree?



It depends on the model. For distance-based models (KNN, K-Means, SVM) or when many features are correlated, yes. Check the correlation matrix first — if there are many highly correlated pairs, PCA will remove redundancy meaningfully. Plot the scree curve. If 20 components capture 95%+ of variance, the reduction is justified. If the model is tree-based (Random Forest, XGBoost), PCA may not help — trees handle high dimensions natively. If interpretability is needed (feature X matters for outcome Y), do NOT use PCA — it destroys interpretability. Always wrap PCA in a Pipeline to prevent data leakage from fitting on the full dataset.

## DAY 23 · Q2      GENERAL MLE ROUND

What does "explained variance" mean in PCA, and how do you decide how many components to keep?

Explained variance ratio = how much of the original data's total variance is captured by each component. PC1 captures the most, PC2 the second most, and so on. They sum to 1.0 across all components. Plot cumulative explained variance vs number of components (scree plot). The standard threshold: keep enough components to explain 95% of the total variance. From the Day 23 notebook: 10 components explained 95% of the Breast Cancer dataset's 30-feature variance. That 10-component model outperformed the full 30-feature model (98.25% vs 97.37%) — PCA removed noise.

## WEEK 3 — ALL QUESTIONS AT A GLANCE

| Day · Company | Question                                                            | Key Points                                                                                             |
|---------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| 17 · General  | Two engineers report different accuracy on the same model and data? | High-variance single tree. Switch to Random Forest. Report CV mean +/- std.                            |
| 17 · Flipkart | Why does Random Forest beat an unconstrained single tree?           | Bagging: each tree overfits differently. Majority vote cancels individual mistakes.                    |
| 18 · General  | 100% train accuracy, 63% test. What's happening?                    | Overfitting. Constrain depth, add regularization, or use Random Forest.                                |
| 18 · General  | How to use a learning curve to decide: more data or new model?      | Large gap = overfit (model also needs to change). Both low = underfit (more capacity needed).          |
| 19 · Paytm    | 98% fraud model accuracy. Business team is happy. Next step?        | Class distribution + confusion matrix + recall on fraud class before celebrating.                      |
| 19 · General  | Recall vs precision: when each?                                     | Recall when false negatives are costly (cancer, fraud). Precision when false alarms are costly (spam). |
| 20 · Google   | Model A: 0.91+/-0.08. Model B: 0.89+/-0.01. Which to deploy?        | Model B. Lower variance = more predictable in production. Std dev matters.                             |
| 20 · Swiggy   | GridSearchCV without leakage — describe the exact setup.            | Wrap ALL preprocessing in Pipeline. CV refits preprocessing per fold. Hold out test set separately.    |
| 21 · Amazon   | Test loss rises after round 60 in XGBoost. What to do?              | early_stopping_rounds — stop at the test loss minimum. Then lower lr + more rounds.                    |
| 21 · General  | When to pick Random Forest over XGBoost?                            | Noisy data, mislabeled rows, fast baseline, explainability needs.                                      |
| 22 · Zomato   | K=5 K-Means — how do you know clusters are meaningful?              | Silhouette > 0.5, visual separation, domain interpretation of each cluster.                            |
| 22 · General  | Why scale before K-Means?                                           | Distance-based: large-range features dominate. Scaling gives each feature equal weight.                |
| 23 · Meta     | 500 features. PCA to 20 components — agree?                         | Check model type, correlation matrix, and scree curve. Trees do not need it. Interpretability lost.    |
| 23 · General  | What is explained variance and how to choose n_components?          | Fraction of total variance captured. 95% threshold. Scree curve elbow. Wrap in Pipeline.               |

## SELF-TEST BEFORE WEEK 4

Without notes: explain in one sentence what boosting does that bagging does not. State what happens to precision and recall when you lower the classification threshold. Describe one scenario where PCA would hurt your model rather than help. Name two things that must live inside a Pipeline to avoid CV data leakage. If any feel shaky, go back to that day.

**Week 4 starts with Neural Networks -> CNNs -> Transfer Learning -> Transformers**

Full notebooks: [github.com/VaishnaviJagtap18/42-Days-of-ML-Challenge](https://github.com/VaishnaviJagtap18/42-Days-of-ML-Challenge) · #42DaysOfML